

Nhập môn kinh điển về phân trị CDQ

AC_Aerolight

28–29 tháng 4, 2013 @ Fuzhou

Nguồn gốc: Discussion topic: divide-and-conquer methods

Others / CDQ divide-and-conquer introduction.pdf

Abstract

Bài giảng trình bày một kiểu phân trị đặc biệt, thường được cộng đồng OI gọi là phân trị CDQ. Tư tưởng chính là chia miền chỉ số thành hai nửa, giải nửa trái trước, xử lý toàn bộ ảnh hưởng của nửa trái lên nửa phải, rồi tiếp tục giải nửa phải. Cách nhìn này biến nhiều bài toán trực tuyến khó xử lý thành các bài toán offline có thể sắp xếp, quét và dùng cấu trúc dữ liệu chuẩn. Nguồn là bộ slide dạng ảnh; bản tiếng Việt này chuyển các slide thành ghi chú bài giảng, giữ mạch ví dụ, các công thức chuyển và các truy hồi đệ phức tạp chính.

Contents

1	Lời nói đầu	2
2	Ví dụ mở đầu: đếm nghịch thế	2
3	NOI2007 Cash	3
3.1	Quy hoạch động ban đầu	3
3.2	Đưa phân trị vào	3
4	WF2011 Machine Works	4
5	BOI2007 Mokia	5
6	Khoảng cách Manhattan động	5
7	Thứ tự từng phần hai và ba chiều	5
7.1	Hai chiều	5
7.2	Ba chiều	6
8	Thứ tự từng phần bốn chiều	6
9	Nhiều chiều và phân trị song song	7
10	K-th trong ma trận con	8

11 ZJOI2013 K-th Query	9
12 Một mẫu bài khác: Package	10
13 HNOI2010 City	10
13.1 Reduction	11
13.2 Contraction	11
13.3 Phân trị theo thao tác	11
14 FJOI2012 Point	11
15 Ghi nhớ khi áp dụng	12

1 Lời nói đầu

Tài liệu gốc nhắc tới một dạng phân trị đặc biệt. Cách làm này được thấy rộng rãi từ bài tập đội tuyển quốc gia năm 2008 của Chen Danqi, nên trong giới OI Trung Quốc nó thường được gọi là phân trị CDQ.

Mục đích của bài giảng là phổ biến phương pháp: các ví dụ được chọn tương đối cơ bản, phần suy luận được viết chậm rãi, và những bài đã quen thuộc có thể được đọc lướt để tập trung vào mẫu tư duy chung.

Một vài công thức truy hồi xuất hiện nhiều lần:

$$T(n) = 2T(n/2) + \mathcal{O}(kn) \Rightarrow T(n) = \mathcal{O}(kn \log n),$$

$$T(n) = 2T(n/2) + \mathcal{O}(kn \log n) \Rightarrow T(n) = \mathcal{O}(kn \log^2 n),$$

$$T(n) = 2T(n/2) + \mathcal{O}(k) \Rightarrow T(n) = \mathcal{O}(kn).$$

Trong đó k là một đa thức không phụ thuộc vào n .

2 Ví dụ mở đầu: đếm nghịch thế

Với một hoán vị P , cần đếm số cặp nghịch thế

$$(i, j) \text{ thỏa } i < j, P_i > P_j.$$

Nếu $|P| \leq 100000$, thuật toán $\mathcal{O}(n^2)$ là quá chậm. Merge sort cho cách làm $\mathcal{O}(n \log n)$:

1. Gọi đệ quy trên nửa trái.
2. Gọi đệ quy trên nửa phải.
3. Đếm các cặp có chỉ số trái nằm ở nửa trái và chỉ số phải nằm ở nửa phải trong lúc trộn hai dãy đã sắp xếp.

Điểm quan trọng là ảnh hưởng nội bộ của từng nửa đã được đệ quy giải quyết; bước gộp chỉ cần quan tâm tới ảnh hưởng chéo.

3 NOI2007 Cash

3.1 Quy hoạch động ban đầu

Có hai loại trái phiếu. Khi mua ở ngày i , tiền được chia theo tỉ lệ $\text{Rate}[i]$ giữa hai loại; khi bán, người chơi bán một tỉ lệ cố định của lượng trái phiếu đang giữ. Biết trước giá hai loại trái phiếu $A[i], B[i]$ trong n ngày và vốn ban đầu S , cần tối đa hóa lợi nhuận. Để tối ưu, mỗi lần giao dịch rõ ràng nên mua hết hoặc bán hết. Ràng buộc chính là

$$1 \leq n \leq 100000.$$

Đặt $F[i]$ là số lượng trái phiếu loại B lớn nhất có thể có sau ngày i . Khi xét giao dịch trước đó là ngày j , công thức trong slide có dạng

$$F[i] = \max \left\{ F[j], \frac{\text{Rate}[j]F[j]A[i] + F[j]B[i]}{\text{Rate}[i]A[i] + B[i]} \right\}.$$

So sánh hai quyết định j, k , ta nhận được điều kiện

$$\frac{\text{Rate}[j]F[j] - \text{Rate}[k]F[k]}{F[j] - F[k]} > -\frac{B[i]}{A[i]}.$$

Về trái thường được gọi là slope(j, k).

Đặt

$$G[i] = \text{Rate}[i]F[i], \quad X_i = (F[i], G[i]).$$

Khi đó slope(j, k) là hệ số góc của đường thẳng qua hai điểm X_j, X_k . Bài toán cần duy trì một tập điểm động hỗ trợ:

1. chèn một điểm bất kỳ trong góc phần tư thứ nhất;
2. với một hệ số góc âm K , tìm đường thẳng có hệ số góc K , đi qua một điểm của tập, và có tung độ cắt trục Y lớn nhất.

Những điểm có thể được chọn ở thao tác thứ hai đều nằm trên bao lồi trên của tập điểm. Nếu xử lý trực tuyến, số biên nhiều và cài đặt dễ rồi.

3.2 Đưa phân trị vào

Quan sát then chốt là hệ số góc truy vấn $-B[i]/A[i]$ chỉ phụ thuộc vào ngày i , không phụ thuộc vào giá trị hiện tại của F . Khi chèn điểm X_i , ta đã biết nó sẽ ảnh hưởng tới những truy vấn $j > i$ nào. Vì vậy có thể định nghĩa

$$\text{Solve}(l, r)$$

là thủ tục tính được các giá trị $F[l], F[l+1], \dots, F[r]$.

Với $\text{mid} = (l+r)/2$, thủ tục có dạng:

1. chạy $\text{Solve}(l, \text{mid})$;
2. xét đồng thời toàn bộ ảnh hưởng của các điểm trong $[l, \text{mid}]$ lên các truy vấn trong $[\text{mid}+1, r]$;
3. chạy $\text{Solve}(\text{mid}+1, r)$.

Như vậy, phần giữa chỉ cần bao lồi của điểm ở nửa trái và các truy vấn ở nửa phải. Bao lồi của hai tập đã sắp xếp có thể gộp tuyến tính; mỗi truy vấn trên bao lồi được trả lời bằng nhị phân. Bước giữa tốn $\mathcal{O}(n \log n)$ trên đoạn độ dài $n = r - l + 1$, do đó

$$T(n) = 2T(n/2) + \mathcal{O}(n \log n) = \mathcal{O}(n \log^2 n).$$

Có thể tiếp tục giảm bước giữa xuống tuyến tính. Thay vì trả lời từng truy vấn bằng nhị phân trên bao lỗi, ta bỏ nhị phân và xử lý offline: sắp xếp bao lỗi điểm cùng toàn bộ truy vấn theo hệ số góc, rồi dùng hai con trỏ quét để tìm điểm tối ưu cho từng truy vấn. Muốn làm được điều đó trong mọi nút đệ quy, cần sắp xếp trước các truy vấn và lưu kết quả của các lần trộn trung gian; nếu đợi đến khi gọi Solve mới trộn thì vẫn lặp lại chi phí sắp xếp.

Chi phí tiền xử lý trộn là $\mathcal{O}(n \log n)$. Trong đệ quy chính, mỗi bước giữa chỉ còn $\mathcal{O}(n)$, nên

$$T(n) = 2T(n/2) + \mathcal{O}(n) = \mathcal{O}(n \log n).$$

Nếu muốn khử đệ quy, slide gợi ý mô phỏng bằng ngăn xếp thủ công hoặc lưu lại các quá trình trung gian. Bài giảng kết luận hai ý: phân trị chỉ xét ảnh hưởng chéo, còn ảnh hưởng nội bộ của mỗi đoạn được đệ quy xử lý; nếu ảnh hưởng nội bộ có thể tạm bỏ qua, bài toán trở thành offline.

4 WF2011 Machine Works

Công ty có quyền sử dụng nhà máy trong N ngày và vốn ban đầu C . Có M máy, mỗi máy có bốn tham số D, P, R, G . Ta có thể mua máy ở ngày D với giá P , từ ngày $D + 1$ máy sinh lời G mỗi ngày, và khi không cần nữa có thể bán máy thu R . Trong nhà máy chỉ được giữ một máy. Không thể vận hành máy vào đúng ngày mua hoặc bán, nhưng có thể bán một máy rồi mua máy khác trong cùng ngày. Sau ngày $N + 1$, phải bán máy đang giữ và cần tối đa hóa vốn.

Dữ liệu có thể tới

$$D \leq 10^9, \quad C \leq 10^9, \quad M \leq 10^5, \\ D_i \leq D, \quad 1 \leq R_i < P_i \leq 10^9, \quad 1 \leq G_i \leq 10^9.$$

Sắp xếp máy theo ngày bán D_i , rồi rạc hóa thời gian, và đặt $F[i]$ là số tiền lớn nhất còn lại sau khi bán máy thứ i . Khi chuyển từ máy j sang máy i :

$$F[i] = \text{Max} (F[i - 1], F[j] - P[j] + R[j] + G[j](D[i] - D[j] - 1)),$$

với điều kiện $F[j] > P[j]$. Đặt

$$A[j] = F[j] - P[j] + R[j] - G[j]D[j] - G[j],$$

ta có

$$F[i] = \text{Max} (F[i - 1], A[j] + G[j]D[i]).$$

Đây lại là truy vấn cực đại trên một tập đường thẳng

$$y = G[j]x + A[j].$$

Nói theo hình học, cần duy trì một tập đường thẳng với hai thao tác:

1. chèn hàm bậc nhất $y = kx + b$;
2. với truy vấn D , lấy giá trị lớn nhất của mọi hàm tại $x = D$.

Slide mô tả đây là việc duy trì một dãy nửa mặt phẳng/bao lỗi tuyến tính. Vì các hoành độ truy vấn $D[i]$ không đổi theo quá trình tính, ta có thể dùng phân trị trên toàn bộ đoạn $[1, n]$ giống bài Cash. Mỗi bước giữa duy trì một họ đường thẳng và trả lời các truy vấn $x = D[i]$, đạt $\mathcal{O}(n \log n)$, cùng bậc với cách duy trì bao lỗi bằng cây cân bằng.

5 BOI2007 Mokia

Bài toán có một bàn cờ rất lớn 2000000×2000000 . Mỗi ô chứa một giá trị $A[x, y]$. Hai thao tác:

$$\text{Add } x \ y \ a : A[x, y] += a,$$

Query $x_0 \ y_0 \ x_1 \ y_1$: hỏi tổng trong hình chữ nhật.

Số thao tác thêm không quá 160000, số truy vấn không quá 10000.

Do kích thước bàn cờ và số truy vấn chênh nhau lớn, cấu trúc dữ liệu hai chiều trực tiếp dễ bị quá bộ nhớ hoặc quá thời gian. Phân trị CDQ xử lý ảnh hưởng của các thao tác Add ở nửa trái lên các truy vấn ở nửa phải. Mỗi truy vấn hình chữ nhật được tách bằng hiệu tiền tố hai chiều:

$$[x_0, x_1] \times [y_0, y_1]$$

được đổi thành bốn truy vấn tiền tố (x, y) . Ở bước gộp, sắp xếp tất cả điểm và truy vấn theo x , quét tăng x , và dùng Fenwick hoặc cây đoạn trên trục y . Khi mỗi lớp phân trị tốn $\mathcal{O}(n \log n)$,

$$T(n) = 2T(n/2) + \mathcal{O}(n \log n) = \mathcal{O}(n \log^2 n).$$

6 Khoảng cách Manhattan động

Tài liệu nêu một bài toán duy trì tập điểm hai chiều P , hỗ trợ:

1. chèn điểm (x, y) ;
2. với truy vấn (x, y) , tìm điểm gần nhất theo khoảng cách Manhattan.

Khoảng cách là

$$\text{dis}(P_1, P_2) = |x_1 - x_2| + |y_1 - y_2|,$$

với $N, m \leq 300000$, $x, y \leq 1000000$. Cây k -d không phù hợp với yêu cầu trong bài giảng.

Loại bỏ dấu giá trị tuyệt đối bằng cách xét bốn hướng: trái trên, trái dưới, phải trên, phải dưới. Chỉ cần mô tả một hướng, ví dụ đáp án nằm ở phía trái dưới truy vấn. Khi đó

$$\text{dis}((x, y), (x', y')) = (x - x') + (y - y') = x + y - (x' + y').$$

Vì vậy cần tìm điểm thỏa $x' \leq x, y' \leq y$ có $x' + y'$ lớn nhất. Nếu không có thao tác chèn động, ta chỉ cần sắp xếp theo x và dùng Fenwick trên y . Với thao tác chèn theo thời gian, phân trị CDQ một lần nữa xử lý ảnh hưởng của các điểm ở nửa trái lên truy vấn ở nửa phải:

1. Solve(l, mid);
2. xử lý ảnh hưởng chéo bằng quét offline theo x và Fenwick theo y ;
3. Solve($\text{mid} + 1, r$).

Mỗi hướng được xử lý tương tự, rồi lấy giá trị nhỏ nhất trong bốn đáp án.

7 Thứ tự từng phần hai và ba chiều

7.1 Hai chiều

Có N người, mỗi người có hai chỉ số năng lực P_i, Q_i . Nếu

$$P_i > P_j \quad \text{và} \quad Q_i > Q_j,$$

nói rằng i mạnh hơn j . Cần tìm dãy dài nhất

$$A = (A_1, A_2, \dots, A_t)$$

sao cho A_i mạnh hơn A_{i+1} . Với $N \leq 100000$, để dễ nhìn ta giả sử P, Q đều là hoán vị của $1, \dots, N$.

Sắp xếp các phần tử theo P_i . Bài toán trở thành LIS trên dãy Q_i :

$$F[i] = \text{Max}\{F[j] \mid j < i, Q[j] < Q[i]\} + 1.$$

Fenwick hoặc cây đoạn cho độ phức tạp $\mathcal{O}(n \log n)$.

7.2 Ba chiều

Với ba chỉ số P_i, Q_i, R_i , sau khi sắp xếp theo P_i , cần tính

$$F[i] = \text{Max}\{F[j] \mid j < i, Q[j] < Q[i], R[j] < R[i]\} + 1.$$

Nếu dùng cây đoạn lồng cây cân bằng hoặc cây đoạn bền vững có thể đạt $\mathcal{O}(n \log^2 n)$. Bài giảng đưa CDQ vào để xử lý chiều chỉ số i :

$$\text{Solve}(l, r) : \begin{cases} \text{Solve}(l, \text{mid}), \\ \text{xử lý ảnh hưởng của } [l, \text{mid}] \text{ lên } [\text{mid} + 1, r], \\ \text{Solve}(\text{mid} + 1, r). \end{cases}$$

Ở bước giữa, xem các điểm bên trái là các điểm có trọng số

$$X = (Q_i, R_i), \quad \text{trọng số } F[i],$$

và với mỗi điểm bên phải (Q_j, R_j) , cần tìm trọng số lớn nhất trong các điểm có

$$Q_i < Q_j, \quad R_i < R_j.$$

Sắp xếp tất cả điểm và truy vấn theo Q , quét theo Q , và dùng Fenwick hoặc cây đoạn để lấy cực đại theo R . Do bước giữa tốn $\mathcal{O}(n \log n)$,

$$T(n) = 2T(n/2) + \mathcal{O}(n \log n) = \mathcal{O}(n \log^2 n).$$

8 Thứ tự từng phần bốn chiều

Với bốn chỉ số P_i, Q_i, R_i, S_i , có hai biến thể trong slide.

Biến thể thứ nhất chỉ cần với mỗi người i tìm xem có người nào mạnh hơn:

$$j < i, \quad Q_j < Q_i, \quad R_j < R_i, \quad S_j < S_i.$$

Bài toán tương đương tìm

$$\text{Min}\{S_j \mid j < i, Q_j < Q_i, R_j < R_i\} < S_i.$$

Sau khi sắp xếp theo P , có thể dùng đúng cách xử lý của bài ba chiều: CDQ theo i , quét theo Q , và cấu trúc dữ liệu trên R để giữ giá trị nhỏ nhất của S .

Biến thể thứ hai yêu cầu dãy dài nhất:

$$F[i] = \text{Max}\{F[j] \mid j < i, Q_j < Q_i, R_j < R_i, S_j < S_i\} + 1.$$

Không còn biến đổi đơn giản như trường hợp chỉ hỏi tồn tại, nên bài giảng dùng hai tầng phân trị. Thủ tục ngoài $\text{Solve}(l, r)$ giải các giá trị $F[l], \dots, F[r]$. Ở giữa, nó gọi một thủ tục phụ $\text{Solve2}(l, r)$ để xử lý ảnh hưởng của $[l, \text{mid}]$ lên $[\text{mid} + 1, r]$ theo ba chiều còn lại.

Trong Solve2 , ta lại phân trị một lần nữa, rồi ở bước giữa của tầng phụ chỉ còn bài toán hai chiều: chèn điểm (R_i, S_i) với trọng số $F[i]$, và truy vấn tại (R_j, S_j) để lấy trọng số lớn nhất trong các điểm thỏa

$$R_i < R_j, \quad S_i < S_j.$$

Phần này được xử lý offline bằng cách sắp xếp và dùng Fenwick/cây đoạn giống trường hợp ba chiều.

Đặt $T_2(n)$ là độ phức tạp của Solve2 :

$$T_2(n) = 2T_2(n/2) + \mathcal{O}(n \log n) = \mathcal{O}(n \log^2 n).$$

Đặt $T(n)$ là độ phức tạp của Solve :

$$T(n) = 2T(n/2) + T_2(n) = 2T(n/2) + \mathcal{O}(n \log^2 n) = \mathcal{O}(n \log^3 n).$$

9 Nhiều chiều và phân trị song song

Slide đặt câu hỏi vui với thứ tự từng phần 100 chiều: mỗi người có 100 chỉ số năng lực $P_{i,1}, P_{i,2}, \dots, P_{i,100}$; nếu mọi chiều của i đều lớn hơn của j thì i mạnh hơn j . Với $N \leq 500$, nếu cứ lồng thêm phân trị hoặc cấu trúc dữ liệu theo từng chiều thì sẽ xuất hiện rất nhiều thừa số $\log n$. Đây là lời nhắc rằng CDQ là một công cụ chứ không phải câu trả lời tự động cho mọi số chiều.

Một ý tưởng khác trong bài giảng là phân trị song song. Ví dụ trong bài POI2011 Meteor có N quốc gia cùng khai thác quỹ đạo tiểu hành tinh và đặt M trạm quan sát. Mỗi trạm thuộc đúng một quốc gia; trạm i kề trạm $i + 1$, còn trạm M kề trạm 1. Trong tương lai xảy ra K trận mưa sao băng. Trận thứ i có đoạn $[L_i, R_i]$ theo chiều kim đồng hồ; nếu $L_i > R_i$ thì đoạn đi qua cuối vòng, tức gồm $L_i, L_i + 1, \dots, M, 1, \dots, R_i$. Mỗi trạm trong đoạn nhận thêm A_i đơn vị thiên thạch. Quốc gia x có mục tiêu W_x ; cần in trận mưa đầu tiên sau đó quốc gia này đạt mục tiêu, hoặc NIE nếu không bao giờ đạt. Slide ghi

$$N, M, K \leq 3 \cdot 10^5, \quad A_i, W_i \leq 10^9.$$

Ví dụ nguồn:

```
3 5
1 3 2 1 3
10 5 7
3
4 2 4
1 3 1
3 5 2
```

kết quả là

```
3
NIE
1
```

Nếu chỉ có một quốc gia, ta có thể dùng cây đoạn có lazy tag để mô phỏng từng trận mưa và kiểm tra các trạm của quốc gia đó. Nhị phân đáp án cho một quốc gia cần mô phỏng $\mathcal{O}(\log K)$ lần. Với nhiều

quốc gia, định nghĩa $\text{Solve}(l, r, S)$ là thủ tục xử lý tập quốc gia S mà đáp án nằm trong $[l, r]$. Sau khi mô phỏng đến mid , chia S thành S_1 đã đạt mục tiêu và S_2 chưa đạt, rồi gọi

$$\text{Solve}(l, \text{mid}, S_1), \quad \text{Solve}(\text{mid} + 1, r, S_2).$$

Cách chia thô phải tính lại quá nhiều lần: phần kiểm tra từng quốc gia tốn $\mathcal{O}(t_x \log M)$, với t_x là số trạm của quốc gia x , nhưng phần mô phỏng các trận mưa có thể dẫn đến truy hồi $T(x) = 2T(x/2) + \mathcal{O}(K \log M)$, chưa đủ tốt.

Cải tiến trong slide là duy trì một cây đoạn toàn cục. Trước khi chạy $\text{Solve}(l, r, S)$, cây chứa hiệu ứng của các trận mưa từ 1 đến $l - 1$; sau khi chạy xong, cây chứa hiệu ứng từ 1 đến r . Ký hiệu $+[l, r]$ là mô phỏng các trận mưa trong đoạn này và $-[l, r]$ là hoàn tác. Thủ tục làm:

$$\begin{aligned} &\text{Solve}(l, r, S) : \\ &\quad +[l, \text{mid}], \\ &\quad \text{chia } S \text{ theo việc đã đủ mục tiêu ở } \text{mid}, \\ &\quad -[l, \text{mid}], \\ &\quad \text{Solve}(l, \text{mid}, S_1), \\ &\quad \text{Solve}(\text{mid} + 1, r, S_2). \end{aligned}$$

Khi đó chi phí mô phỏng theo chiều đáp án là

$$T(x) = 2T(x/2) + \mathcal{O}(x \log M) = \mathcal{O}(x \log x \log M),$$

và toàn bộ phương pháp đạt dạng $\mathcal{O}(K \log K \log M + N \log K \log M)$, đủ qua.

Nhìn theo phân trị song song, ban đầu miền đáp án của mỗi quốc gia là $[1, K]$. Ta đồng thời nhị phân đáp án của tất cả quốc gia:

1. lấy trung điểm mid_x của miền đáp án của quốc gia x ;
2. sắp xếp các mid_x , rồi xử lý offline các trận mưa sao theo thứ tự thời gian;
3. nếu tại mid_x lượng thu được đủ W_x , đẩy miền về nửa trái; ngược lại đẩy về nửa phải.

Sau $\log k$ vòng, mỗi miền đáp án có độ dài 1. Slide ghi độ phức tạp theo dạng

$$\log k \cdot (k \log m + n \log n).$$

10 K-th trong ma trận con

Một ví dụ khác trong slide là bài toán truy vấn thứ tự trên ma trận: cho ma trận A kích thước $n \times n$, có q truy vấn

$$(x_1, y_1, x_2, y_2, k),$$

hỏi số nhỏ thứ k trong hình chữ nhật $[x_1, y_1]$ đến $[x_2, y_2]$. Ràng buộc minh họa là

$$n \leq 500, \quad q \leq 60000,$$

và cho phép xử lý offline.

Với một truy vấn đơn lẻ, có thể nhị phân đáp án. Với một giá trị mid , đặt

$$D_{i,j} = [A_{i,j} \leq \text{mid}],$$

rồi dùng tổng tiền tố hai chiều để kiểm tra trong hình chữ nhật có ít nhất k phần tử không vượt quá mid hay không.

Để xử lý nhiều truy vấn, định nghĩa $\text{Solve}(l, r, S)$ là thủ tục xử lý tập truy vấn S mà đáp án nằm trong miền giá trị $[l, r]$. Tách tại mid , dùng cấu trúc dữ liệu để tính với từng truy vấn có bao nhiêu phần tử $\leq mid$, rồi đưa truy vấn sang nửa trái hoặc nửa phải. Nếu dùng trực tiếp cây Fenwick hai chiều, mỗi thao tác có thể xem như $\mathcal{O}(\log^2 n)$, dẫn đến truy hồi dạng

$$F(n) = 2F(n/2) + \mathcal{O}(q \log^2 n).$$

Slide cũng nhắc cách tối ưu để tránh cấu trúc dữ liệu hai chiều: biến các ô được chèn và các truy vấn hình chữ nhật thành các sự kiện, sắp xếp/ghép theo tọa độ x , rồi chỉ cần một cây Fenwick theo tọa độ y . Khi đó bước giữa còn $\mathcal{O}(n \log n)$ theo số sự kiện của tầng đang xét, và độ phức tạp toàn cục giảm một thừa số $\log n$.

11 ZJOI2013 K-th Query

Bài toán có n vị trí và m thao tác. Có hai loại thao tác:

- 1 a b c : thêm một số c vào mọi vị trí từ a đến b ;
- 2 a b c : hỏi trong tất cả các số đang nằm ở các vị trí từ a đến b , số lớn thứ c là bao nhiêu.

Các ràng buộc trong slide là $n, m \leq 50000$; với thao tác thêm thì $c \leq n$, còn trong truy vấn c có thể lớn tới miền số nguyên dài.

Cách dùng cấu trúc dữ liệu trực tiếp là cây đoạn lồng cây cân bằng: tầng ngoài theo giá trị được chèn, tầng trong theo tọa độ vị trí, thường cần cập phát động. Với một truy vấn đơn, ta lại có thể nhị phân đáp án và duy trì cây đoạn hoặc Fenwick để kiểm tra số phần tử trong đoạn.

CDQ theo miền giá trị cho ta một cách làm offline. Gọi $\text{Solve}(l, r, S)$ xử lý mọi thao tác/truy vấn trong S có đáp án thuộc $[l, r]$. Ở giữa mid , chỉ các thao tác thêm có $c \leq mid$ mới ảnh hưởng tới câu hỏi “có bao nhiêu số không vượt quá mid trong đoạn $[a, b]$?”. Ta sắp các thao tác theo thời gian và dùng Fenwick theo vị trí để cộng/xóa các đoạn tác động, rồi chia S sang hai nửa.

Sau khi phân bổ chi phí mỗi thao tác ở một tầng, bước giữa ở mức $\mathcal{O}(n \log n)$, nên truy hồi vẫn có dạng quen thuộc

$$T(n) = 2T(n/2) + \mathcal{O}(n \log n) = \mathcal{O}(n \log^2 n).$$

Ba ví dụ liên tiếp trong phần này có cùng điểm chung: biến câu hỏi thứ tự thành nhiều phép kiểm tra theo ngưỡng, rồi dùng phân trị trên miền đáp án để xử lý toàn bộ truy vấn offline.

Slide mẫu cho dữ liệu:

```
2 5
1 1 2 1
1 1 2 2
2 1 1 2
2 1 1 1
2 1 2 3
```

và kết quả:

```
1
2
1
```

Sau thao tác đầu, vị trí 1 và 2 đều chỉ có số 1. Sau thao tác thứ hai, hai vị trí đó đều có 1, 2. Truy vấn thứ ba hỏi số lớn thứ 2 trong vị trí 1, đáp án 1; truy vấn thứ tư hỏi số lớn thứ 1 trong vị trí 1, đáp án 2; truy vấn thứ năm hỏi số lớn thứ 3 trong đoạn $[1, 2]$, đáp án 1.

12 Một mẫu bài khác: Package

Với bài *Package*, có N vật, vật i có trọng lượng W_i và giá trị G_i ; mỗi vật chỉ được lấy một lần. Có Q truy vấn, mỗi truy vấn là (X, i) : với ba lô dung lượng tối đa X , dùng tất cả vật trừ vật thứ i , hỏi tổng giá trị lớn nhất đạt được. Ràng buộc trong slide là

$$N \leq 100, \quad X \leq 10000, \quad Q \leq 1000000.$$

Nếu chỉ có một truy vấn, hoặc mọi truy vấn có cùng i , ta chạy quy hoạch động ba lô 0/1 trong $\mathcal{O}(nm)$, với m là dung lượng. Đưa một vật vào trạng thái ba lô hiện tại tốn $\mathcal{O}(m)$, nên ký hiệu $+ [l, r]$ là thêm các vật $l, \dots, r$ vào ba lô, chi phí $\mathcal{O}((r - l + 1)m)$.

Định nghĩa $\text{Solve}(l, r)$ để xử lý mọi truy vấn có chỉ số $l \leq i \leq r$. Thủ tục mở rộng có thêm trạng thái S :

$$\text{Solve}(l, r, S),$$

trong đó S là một ba lô 0/1 đã chứa mọi vật nằm ngoài đoạn $[l, r]$. Khi tới lá $\text{Solve}(L, L, S)$, ta trả lời tất cả truy vấn $i = L$.

Đệ quy:

$$\text{Solve}(l, r, S) : \begin{cases} \text{Solve}(l, \text{mid}, S + [\text{mid} + 1, r]), \\ \text{Solve}(\text{mid} + 1, r, S + [l, \text{mid}]). \end{cases}$$

Nếu mỗi lần gộp trạng thái ba lô tốn $\mathcal{O}(nm)$, truy hồi là

$$T(n) = 2T(n/2) + \mathcal{O}(nm) = \mathcal{O}(nm \log n).$$

Đây là một ví dụ khác của cùng khuôn: tại mỗi nhánh, ta đưa toàn bộ ảnh hưởng của phần còn lại vào trạng thái, để câu hỏi tại lá trở nên trực tiếp.

13 HNOI2010 City

Bài toán cho đồ thị vô hướng n đỉnh, m cạnh, mỗi cạnh có trọng số w_i . Có q thao tác (E_i, Z_i) , nghĩa là đổi trọng số cạnh thứ E_i thành Z_i . Sau mỗi thao tác cần in trọng lượng cây khung nhỏ nhất hiện tại. Ràng buộc trong slide:

$$n \leq 20000, \quad m \leq 50000, \quad q \leq 50000, \quad w_i, Z_i \leq 10^8.$$

Nếu trọng số của một cạnh chỉ giảm, khi cập nhật cạnh (x, y) , cây khung nhỏ nhất hoặc không đổi, hoặc cạnh (x, y) được thêm vào và một cạnh lớn nhất trên đường đi từ x đến y trong cây cũ bị loại. Khi đó có thể duy trì cây khung bằng Link-Cut Tree: thêm một cạnh, xóa một cạnh, và hỏi cạnh lớn nhất trên đường đi. Trường hợp trọng số chỉ tăng là đối xứng hơn nhưng vẫn khó nếu muốn xử lý trực tuyến đơn giản.

Ở đây lợi thế là toàn bộ dãy thao tác đã biết trước. Ta phân trị trên thời gian. Gọi S là tập cạnh bị sửa trong đoạn thao tác đang xét. Trước khi đệ quy xuống đoạn con, ta muốn thu nhỏ đồ thị mà vẫn giữ đúng mọi đáp án trong đoạn đó. Slide dùng hai phép: **reduction** và **contraction**.

13.1 Reduction

Cho đồ thị có trọng số G và một tập cạnh S . Đặt trọng số các cạnh trong S thành $+\infty$, rồi lấy cây khung nhỏ nhất T của G . Khi đó dù trọng số các cạnh trong S thay đổi thế nào, cây khung nhỏ nhất của G không cần dùng bất kỳ cạnh nào ngoài $T \cup S$. Thật vậy, với một cạnh $e \notin T \cup S$, hai đầu của e nối với nhau bằng một đường trong T . Các cạnh trên đường này đều không thuộc S , nên trọng số của chúng không đổi khi ta thay đổi S ; theo tính chất chu trình của MST, e không thể trở thành cạnh cần thiết.

Vì vậy ta có thể xóa mọi cạnh ngoài $T \cup S$. Số cạnh còn lại không vượt quá $n + |S| - 1$. Bước này gọi là reduction, mục tiêu là giảm số cạnh. Chi phí cùng bậc với chạy MST, thường viết $\mathcal{O}(m \log m)$.

13.2 Contraction

Ngược lại, đặt trọng số các cạnh trong S thành $-\infty$, rồi lấy cây khung nhỏ nhất T . Khi đó dù trọng số các cạnh trong S thay đổi thế nào, mọi cạnh trong $T \setminus S$ chắc chắn thuộc cây khung nhỏ nhất cuối cùng. Có thể hình dung tăng dần trọng số từng cạnh của S : mỗi lần thay đổi, cây khung hoặc không đổi, hoặc một cạnh trong S bị thay thế; các cạnh ngoài S đã được chọn trong T không bị loại ra theo quá trình này.

Do đó ta có thể co các cạnh trong $T \setminus S$ thành các thành phần liên thông và cập nhật lại danh sách cạnh theo thành phần. Bước này gọi là contraction, mục tiêu là giảm số đỉnh. Sau contraction, số đỉnh còn khoảng $|S| + 1$.

13.3 Phân trị theo thao tác

Từ hai phép trên, với một đồ thị G và tập thao tác S của một đoạn thời gian, ta có thể trong $\mathcal{O}(m \log m)$ rút đồ thị về số cạnh không quá $n + |S| - 1$ và số đỉnh không quá $|S| + 1$ mà vẫn giữ nguyên kết quả trong đoạn đang xét.

Định nghĩa $\text{Solve}(l, r, G)$ để xử lý các thao tác từ l đến r trên đồ thị đã được rút gọn phù hợp với đoạn đó:

```
Solve( $l, r, G$ ) :  
    reduction( $G$ );  
    contraction( $G$ );  
    Solve( $l, \text{mid}, G$ );  
    Solve( $\text{mid} + 1, r, G$ );  
    recover( $G$ ).
```

Tại lá, sau một thao tác, đồ thị đã nhỏ tới mức chỉ còn trường hợp cơ bản; có thể trực tiếp xét hai đỉnh và một cạnh tương ứng để cập nhật đáp án. Trong mỗi lời gọi $\text{Solve}(l, r, G)$, sau reduction–contraction, số đỉnh và số cạnh của G đều là $\mathcal{O}(r - l)$. Vì vậy chi phí mỗi tầng là $\mathcal{O}(n \log n)$ theo kích thước đoạn, và truy hồi tổng quát là

$$T(n) = 2T(n/2) + \mathcal{O}(n \log n).$$

14 FJOI2012 Point

Slide cuối nêu thêm một bài gợi ý. Có một mảng độ dài n , ban đầu rỗng. Thực hiện n thao tác (P_i, Q_i) , nghĩa là điền giá trị Q_i vào vị trí P_i . Hai dãy P_i và Q_i đều là hoán vị của $1, \dots, n$. Sau mỗi thao tác, cần in số nghịch thế của dãy hiện tại; $n \leq 50000$.

Các cấu trúc như cây lồng cây hoặc danh sách khối có thể đạt gần $\mathcal{O}(n \log^2 n)$, nhưng trong kỳ thi chúng không dễ viết và không phải lúc nào cũng đạt trọn điểm. Bài giảng dùng câu hỏi này để nhấn

mạnh tinh thần của CDQ: nếu tìm được một trục thời gian phù hợp, ta có thể hy vọng giảm bớt một thừa số $\log n$ bằng cách đưa các truy vấn động về xử lý offline.

15 Ghi nhớ khi áp dụng

Các ví dụ trong tài liệu có bề ngoài rất khác nhau, nhưng cùng dựa trên một chuỗi quyết định:

1. Xác định trục thời gian hoặc trục phụ thuộc: thao tác ở nửa trái có thể ảnh hưởng tới truy vấn hoặc trạng thái ở nửa phải.
2. Đảm bảo ảnh hưởng nội bộ đã do đệ quy phụ trách.
3. Biến bước ảnh hưởng chéo thành một bài toán offline: sắp xếp theo một chiều, quét, rồi dùng Fenwick, cây đoạn, bao lồi hoặc cấu trúc dữ liệu phù hợp cho các chiều còn lại.
4. Phân tích truy hồi của bước giữa. Nếu bước giữa tốn $\mathcal{O}(n \log^c n)$, thường nhận thêm một thừa số $\log n$ ở tầng CDQ.